



The Inverse Problem in Science

— Bayesian Statistics & Implicit-Likelihood Methods —

Christopher Eckner (ceckner@iac.es)

UNDARK Postdoctoral Researcher

Instituto de Astrofísica de Canarias, Tenerife, Spain

Course: AI4Science, winter term 2026

13th of March 2026 | University of Nova Gorica, Slovenia

Lecture II Outline

Recap

Bayesian framework, ABC, three SBI flavours

Part I

Autoregressive models and their connection to normalising flows

Part II

Sequential SBI methods

Part III

Diagnostics and validation

Part IV

Good practices and practical workflow

Part V

Examples of SBI in practice

Recap: The Inverse Problem and Bayes' Theorem

Goal: Given observed data $\mathbf{x}_0$, infer plausible model parameters Θ .

$$p(\Theta | \mathbf{x}) = \frac{p(\mathbf{x} | \Theta) p(\Theta)}{p(\mathbf{x})}$$

SBI setting: We can *sample* $(\mathbf{x}, \Theta) \sim p(\mathbf{x}|\Theta) p(\Theta)$ by running a forward simulator, but we *cannot evaluate* the likelihood $p(\mathbf{x}|\Theta)$.

Method	Needs likelihood?	Needs simulations?
MCMC / Nested Sampling	✓	✗
HMC / Variational Inference	✓ (+ gradients)	✗
SBI	✗	✓

Recap: The Three Flavours of Neural SBI

	NPE	NLE	NRE
Learns	$q_{\phi}(\Theta \mathbf{x}) \approx p(\Theta \mathbf{x})$	$q_{\phi}(\mathbf{x} \Theta) \approx p(\mathbf{x} \Theta)$	$r(\Theta; \mathbf{x}) \approx p(\Theta \mathbf{x})/p(\Theta)$
Loss	Neg. log-prob	Neg. log-prob	Binary cross-entropy
Network	Normalising flow	Normalising flow	MLP / ResNet
Posterior	Direct forward pass	MCMC	MCMC / reweighting
Amortised?	✓✓	✓	✓

Key shared advantages:

- **Amortisation:** train once, infer for any $\mathbf{x}_0$ without new simulations.
- **Trivial marginal inference:** exclude nuisance parameters from the network; the data implicitly marginalises over them.
- **Learned compression:** embedding networks $s_{\psi}(\mathbf{x})$ trained end-to-end replace hand-crafted summary statistics.

Autoregressive Models for Posterior Density Approximation

The Chain Rule of Probability — A Universal Decomposition

Remember from last lecture: *Any* joint density over D variables can be decomposed exactly:

$$p(\mathbf{x}) = p(x_1) \prod_{i=2}^D p(x_i \mid x_1, \dots, x_{i-1})$$

Each factor is a **one-dimensional conditional density** that depends only on the variables that come *before* it in the chosen ordering.

This is not an approximation. It is an exact identity from probability theory.

An autoregressive model parameterises each conditional separately:

$$p(x_i \mid x_{1:i-1}) = \text{SomeFamily}\left(x_i; \underbrace{\mu_i(x_{1:i-1}), \sigma_i(x_{1:i-1})}_{\text{parameters computed from past}}\right)$$

The functions μ_i, σ_i can be neural networks — their only constraint is that x_i 's parameters depend *only on* $x_1, \dots, x_{i-1}$, never on future variables.

The Autoregressive Principle Is Everywhere

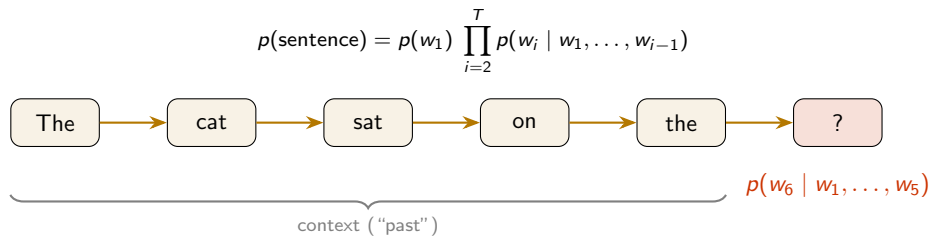
The same factorisation appears across very different domains:

Domain	“Variables” x_i	Model family
Time series	Observations at times $t_1, t_2, \dots$	ARIMA, state-space models, RNNs
Image generation	Pixels in raster order	PixelRNN, PixelCNN (van den Oord et al., 2016)
Audio synthesis	Waveform samples	WaveNet (van den Oord et al., 2016)
Language models	Tokens in a sentence	GPT, LLaMA, ... (LLMs)
Density estimation	Components of $\mathbf{x} \in \mathbb{R}^D$	MADE, MAF (this lecture)

In every case, the model predicts the **distribution of the next element** given all previous ones. The differences lie in *what* the elements are and *how* the conditionals are parameterised.

Example: Language Models as Autoregressive Models

A Large Language Model (LLM) generates text one **token** at a time:



PixelRNN / **PixelCNN** does the same, but with pixels:

$$p(\text{image}) = \prod_{i=1}^D p(\text{pixel}_i \mid \text{pixel}_1, \dots, \text{pixel}_{i-1})$$

Pixels are ordered in raster scan (top-left to bottom-right). Each conditional predicts RGB values given all previously generated pixels.

Key insight for this lecture: The same autoregressive principle underlies **MAF**, the flow architecture we use for density estimation in SBI.

An Autoregressive Model Defines a Transformation

Consider Gaussian conditionals (Papamakarios et al., 2017):

$$p(x_i | x_{1:i-1}) = \mathcal{N}(x_i; \mu_i, (e^{\alpha_i})^2) \quad \text{where} \quad \mu_i = f_\mu^{(i)}(x_{1:i-1}), \quad \alpha_i = f_\alpha^{(i)}(x_{1:i-1})$$

Sampling proceeds sequentially using standard normal noise $u_i \sim \mathcal{N}(0, 1)$:

Forward transformation: $\mathbf{u} \rightarrow \mathbf{x}$

$$x_i = u_i \cdot e^{\alpha_i} + \mu_i \quad \text{where} \quad \mu_i = f_\mu^{(i)}(x_{1:i-1}), \quad \alpha_i = f_\alpha^{(i)}(x_{1:i-1})$$

This is a **transformation** $\mathbf{x} = f(\mathbf{u})$ from i.i.d. noise $\mathbf{u} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ to data $\mathbf{x}$.

Crucially:

- The Jacobian $\partial \mathbf{x} / \partial \mathbf{u}$ is **lower-triangular** by the autoregressive structure (each x_i depends on u_i and on $x_{1:i-1}$, which in turn depend on $u_{1:i-1}$).
- The determinant of a triangular matrix is the product of diagonal entries:

$$\left| \det \frac{\partial \mathbf{x}}{\partial \mathbf{u}} \right| = \prod_i e^{\alpha_i} \implies \log \left| \det \frac{\partial \mathbf{x}}{\partial \mathbf{u}} \right| = \sum_i \alpha_i$$

$\Rightarrow$ This transformation is an **invertible map with a tractable Jacobian**: a normalising flow!

Inverting the Transformation: From $\mathbf{x}$ to $\mathbf{u}$

Given a datapoint $\mathbf{x}$, recover the noise $\mathbf{u}$ that would have generated it:

Inverse transformation: $\mathbf{x} \rightarrow \mathbf{u}$

$$u_i = (x_i - \mu_i) e^{-\alpha_i} \quad \text{where} \quad \mu_i = f_{\mu}^{(i)}(x_{1:i-1}), \quad \alpha_i = f_{\alpha}^{(i)}(x_{1:i-1})$$

The density of $\mathbf{x}$ follows from the normalising flow formula:

$$\log p(\mathbf{x}) = \underbrace{\log \pi_{\mathbf{u}}(\mathbf{u})}_{\text{standard Gaussian}} + \underbrace{\sum_i (-\alpha_i)}_{\log |\det \partial f^{-1} / \partial \mathbf{x}|} = -\frac{D}{2} \log(2\pi) - \frac{1}{2} \sum_i u_i^2 - \sum_i \alpha_i$$

Computational asymmetry:

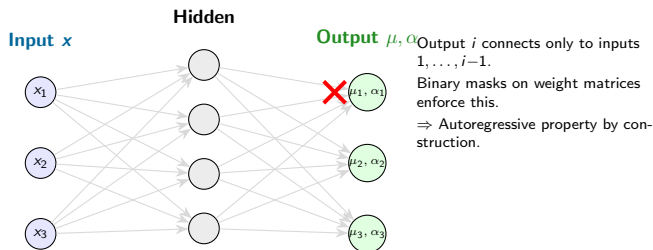
- Computing $\mathbf{u}$ from $\mathbf{x}$ (inverse): all μ_i, α_i depend on $x_{1:i-1}$, which are *given* $\Rightarrow$ can be computed **in parallel** using masking.
- Computing $\mathbf{x}$ from $\mathbf{u}$ (sampling): x_i depends on $x_{1:i-1}$, which must be computed first $\Rightarrow$ requires D **sequential** steps.

$\Rightarrow$ Density evaluation is fast, sampling is slow. This is the **MAF regime**.

MADE: Parallelising the Autoregressive Property

The functions $\{f_{\mu}^{(i)}, f_{\alpha}^{(i)}\}$ could be D separate networks, but that's expensive.

MADE (Masked Autoencoder for Distribution Estimation, Germain et al. 2015) uses a *single* feed-forward network with **masked weight matrices** to compute all μ_i, α_i in one pass:



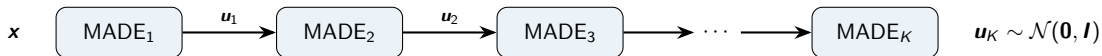
Result: computing all D conditionals costs **one forward pass** — fully parallelisable on GPUs.

MAF: Stacking MADEs into a Deep Flow

A single MADE with Gaussian conditionals has limited flexibility (each conditional is unimodal). The MAF insight (Papamakarios et al., 2017):

Masked Autoregressive Flow (MAF)

Stack K MADE layers. Each layer models the “random numbers” $\mathbf{u}$ of the previous layer. The result is a K -step normalising flow.



Each layer can use a different variable ordering, breaking symmetry and increasing expressiveness. After K layers, even Gaussian conditionals can capture multimodal joint densities.

Density evaluation: one forward pass per MADE $\Rightarrow K$ parallel passes total.

Sampling: D sequential steps per layer $\Rightarrow K \times D$ sequential steps total.

MAF vs. IAF: A Computational Trade-Off

	MAF	IAF
μ_i, α_i depend on	$x_{1:i-1}$ (data)	$u_{1:i-1}$ (noise)
Density $p(\mathbf{x})$	One pass (fast)	D sequential passes (slow)
Sampling $\mathbf{x} \sim p$	D sequential passes (slow)	One pass (fast)
Best for	Density estimation, SBI	Variational inference (only own samples)

Why MAF for SBI?

In NPE, the training loss $\mathcal{L} = -\mathbb{E}[\log q_\phi(\Theta|\mathbf{x})]$ requires **evaluating the density** at externally provided training pairs $(\mathbf{x}, \Theta)$ — exactly where MAF excels.

Sampling only happens at inference time (once, for a given observation), so the slow sequential generation is acceptable.

Real NVP: A Special Case of Both MAF and IAF

The **coupling layer** (Dinh et al., 2017) splits input into two groups:

$$\begin{aligned}x_{1:d} &= u_{1:d} && \text{(copied unchanged)} \\x_{d+1:D} &= u_{d+1:D} \odot e^{\alpha} + \mu && \text{where } \mu = f_{\mu}(u_{1:d}), \alpha = f_{\alpha}(u_{1:d})\end{aligned}$$

This is a **special case** of the MAF autoregressive transform where:

- Variables $1, \dots, d$: set $\mu_i = 0, \alpha_i = 0$ (identity).
- Variables $d+1, \dots, D$: make μ_i, α_i depend only on $x_{1:d}$ (not all of $x_{1:i-1}$).

Advantage: Both density evaluation *and* sampling are a single forward pass.

Disadvantage: Less flexible — each variable is only influenced by its “group partner”, not by all predecessors.

Hierarchy of generality:

$$\text{Real NVP (coupling)} \subset \text{MAF (full autoregressive)} \subset \text{general flow}$$

The Flexibility Bottleneck

In MAF/Real NVP, each element is transformed by an **affine map**:

$$x_i = u_i \cdot e^{\alpha_i} + \mu_i$$

This is an invertible, differentiable, monotonic transformation — but it can only **scale and shift**. A single affine map cannot produce nonlinear warping.

Stacking many layers compensates, but each layer individually is limited.

The question

Can we replace the affine elementwise transformation with something **more expressive**, while keeping invertibility, tractable Jacobians, and differentiability?

Answer (Durkan, Bekasov, Murray & Papamakarios, 2019): **Monotonic rational-quadratic splines**.

Rational-Quadratic Splines: The Idea

Construction:

- 1 Define K **bins** on an interval $[-B, B]$ via $K+1$ “knot” points (x_k, y_k) .
- 2 At each internal knot, specify a **positive derivative** $\delta_k > 0$.
- 3 Within each bin, fit a **rational-quadratic segment**:

$$g(\xi) = \frac{a\xi^2 + b\xi + c}{d\xi^2 + e\xi + f}$$

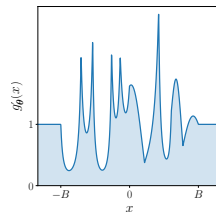
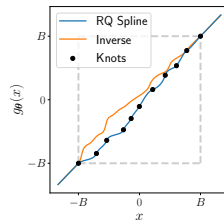
where ξ is the local coordinate and a, b, c, d, e, f are determined by the knot positions and derivatives.

- 4 Outside $[-B, B]$: identity (linear tails).

By construction, g is **monotonically increasing**, **differentiable**, and **analytically invertible** (inversion = solving a quadratic equation).

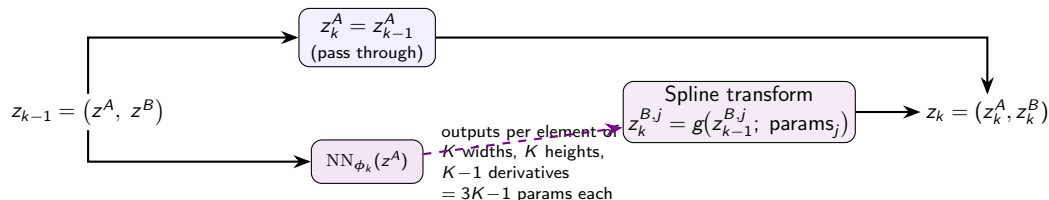
[See C. Durkan *et al.*: *Neural Spline Flows* (2019)]

More on the different normalising flow schemes next week.



Neural Spline Flows: Putting It Together

NSF = any coupling or autoregressive flow where the affine elementwise map is replaced by a rational-quadratic spline.



Compared to affine coupling:

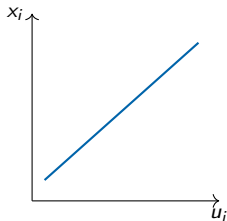
	Affine (MAF/Real NVP)	Spline (NSF)
Elementwise map	$x_i = u_i e^{\alpha_i} + \mu_i$	$x_i = g(u_i; \text{knots, derivs})$
NN outputs per element	$2 (\mu_i, \alpha_i)$	$3K-1$ (knots + derivatives)
Nonlinearity per layer	None (just scale + shift)	Piecewise rational-quadratic
Invertible?	Trivially	Analytic (solve a quadratic)
Jacobian	e^{α_i}	$g'(u_i)$ in closed form

NSF is strictly more expressive per layer $\Rightarrow$ fewer layers needed for the same quality $\Rightarrow$ currently the preferred architecture for NPE in SBI.

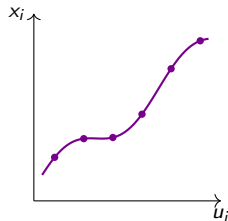
Why Splines Work: Intuition

An affine map can only **stretch uniformly**: one scale, one shift per element.

A spline with K bins can stretch **differently in different regions**:



Affine
one slope everywhere



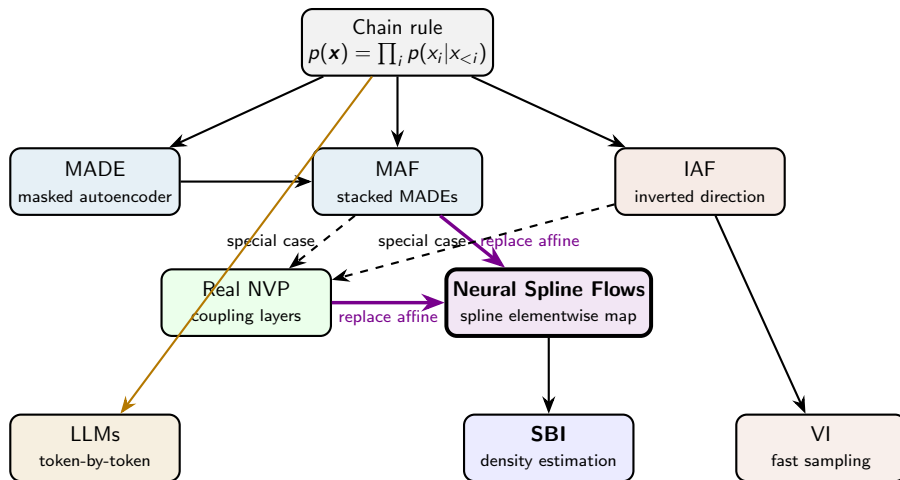
Spline ($K=6$)
variable slope per bin

Where the spline is steep, density is **compressed** (low $p(x_i)$).

Where the spline is flat, density is **expanded** (high $p(x_i)$).

With K bins, a single layer can create K regions of different stretching — capturing multi-modal structure that an affine map could never achieve in one step.

The Architecture Landscape



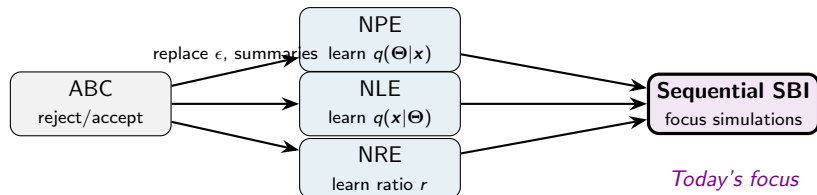
The **same autoregressive principle** underlies LLMs (discrete tokens), PixelRNN (pixels), WaveNet (audio), and flow-based density estimators (continuous $\mathbf{x}$). The difference is in what is modelled and how the conditionals are parameterised.

Summary

- 1 **The autoregressive principle** decomposes any joint density via the chain rule. It appears in time series, image generation (PixelRNN), language models (LLMs), and density estimation (MADE/MAF).
- 2 **An autoregressive model is a normalising flow**: sampling introduces noise $\mathbf{u} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and transforms it autoregressively into data $\mathbf{x}$. The triangular Jacobian makes the determinant trivially computable.
- 3 **MAF** stacks MADE layers to increase flexibility, with fast density evaluation (one pass) but slow sampling (D sequential steps). **IAF** reverses this trade-off.
- 4 **Real NVP** is a special case of both MAF and IAF (coupling instead of full autoregressive), gaining fast density *and* sampling at the cost of expressiveness.
- 5 **Neural Spline Flows** replace the affine elementwise map $(\mu + \sigma u)$ with a monotonic rational-quadratic spline whose $3K - 1$ parameters per element are output by the neural network. This dramatically increases per-layer expressiveness while preserving analytic invertibility.
- 6 For SBI, **NSF + coupling/autoregressive layers** is currently the standard architecture for NPE density estimators.

Sequential Simulation-Based Inference

Recap: From ABC to Neural SBI



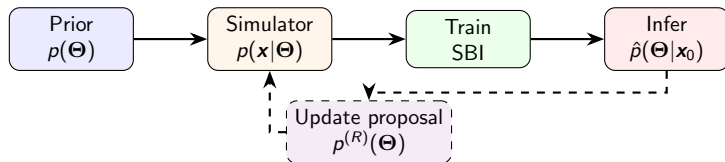
Limitation of single-round (amortised) SBI: Simulations are drawn from the full prior $p(\Theta)$, wasting budget on parameter regions irrelevant for a specific observation $\mathbf{x}_0$.

Sequential SBI addresses this by adaptively focusing simulations on regions consistent with $\mathbf{x}_0$.

Why Sequential? The Simulation Efficiency Problem

For a specific target observation $\mathbf{x}_0$, most prior samples $\Theta \sim p(\Theta)$ produce simulations far from $\mathbf{x}_0$.

Sequential approaches replace the prior with an adaptive **proposal distribution** $p^{(R)}(\Theta)$ that progressively concentrates on regions consistent with $\mathbf{x}_0$:



Two main strategies:

- 1 **Prior truncation:** restrict the prior support to high-posterior regions.
- 2 **Proposal-based:** use the previous round's posterior as the proposal (SNPE-A/B/C).

Prior Truncation

Iteratively discard low-density regions discovered in previous rounds.

Box truncation (Miller et al., 2022)

Threshold each 1D marginal posterior at a small fraction ϵ of its peak:

$$\tilde{p}^{(R)}(\boldsymbol{\Theta}) = \frac{1}{Z} \prod_{j=1}^d \mathbb{I}\left(\theta_j \in \Gamma_{\theta_j}^{(R-1)}\right) p(\boldsymbol{\Theta})$$

where $\Gamma_{\theta_j}^{(R-1)}$ is the support region from the previous round for the j -th parameter.

Simple to implement but can be inefficient with strong parameter correlations (includes excessive low-density volume).

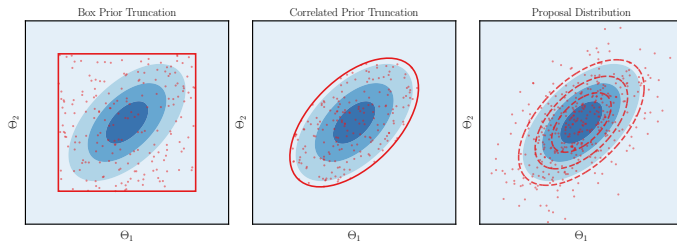
Correlated truncation (Deistler et al., 2022; Anau Montel et al., 2024)

Truncate to the highest-probability region containing $1 - \epsilon$ of the posterior mass:

$$\tilde{p}^{(R)}(\boldsymbol{\Theta}) = \frac{1}{Z} \mathbb{I}\left(\boldsymbol{\Theta} \in \Gamma_{\boldsymbol{\Theta}}^{(R-1)}\right) p(\boldsymbol{\Theta})$$

Tighter confinement to high-density regions; better for correlated posteriors.

Visualising Proposal Strategies



Strategy

Pros

Cons

Box truncation

Simple, locally amortised

Inefficient with correlations

Correlated truncation

Tighter, locally amortised

Requires density evaluation

Learned proposal ($\approx$ posterior)

Most focused

Breaks amortisation; needs correction

SNPE-A: Post-Hoc Correction

Idea: Use the posterior from round $R-1$ as the proposal for round R :

$$\tilde{p}^{(R)}(\Theta) = q_{\phi}^{(R-1)}(\Theta \mid \mathbf{x}_0)$$

Train on the standard NPE loss (Eq. 9 from Lecture I). The learned $q_{\phi}^{(R)}$ approximates a *proposal-biased* posterior:

$$q_{\phi}^{(R)}(\Theta \mid \mathbf{x}) \simeq \frac{1}{Z} p(\mathbf{x} \mid \Theta) \tilde{p}^{(R)}(\Theta)$$

Correction at test time:

$$p(\Theta \mid \mathbf{x}) \propto q_{\phi}^{(R)}(\Theta \mid \mathbf{x}_0) \frac{p(\Theta)}{\tilde{p}^{(R)}(\Theta)}$$

Limitations:

- Requires Gaussian or mixture-density proposals for closed-form correction.
- Gaussian proposals struggle with multimodality or heavy tails.
- Can induce mode collapse when proposal has weak overlap with true posterior.

SNPE-B: Importance-Weighted Training

Idea: Correct for proposal bias *during training* via importance weights:

SNPE-B loss (Lueckmann et al., 2017)

$$\mathcal{L}^{\text{SNPE-B}}[\Phi] = -\mathbb{E}_{\Theta \sim \tilde{p}, \mathbf{x} \sim p(\mathbf{x}|\Theta)} \left[\frac{p(\Theta)}{\tilde{p}(\Theta)} \log q_{\Phi}^{\text{NPE}}(\Theta|\mathbf{x}) \right]$$

The importance weights $p(\Theta)/\tilde{p}(\Theta)$ reweight each training sample to correct for the mismatch between proposal and prior.

Result: The learned q_{Φ}^{NPE} directly approximates the true posterior — no test-time correction needed.

Advantage: Removes restriction to Gaussian proposals; can use normalising flows.

Disadvantage: Importance weights can have high variance $\Rightarrow$ unstable training.

SNPE-C (APT): Best of Both Worlds

Automatic Posterior Transformation (Greenberg et al., 2019) avoids both the architecture restrictions of SNPE-A and the high-variance weights of SNPE-B.

Key innovation: “Atomic” proposals — train on proposal sets sampled from a hyper-proposal distribution, ensuring the network learns the correct posterior despite proposal bias.

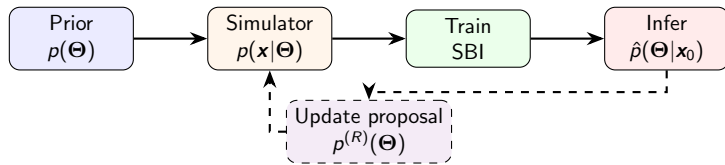
Advantages of SNPE-C:

- No restriction on density estimator architecture (can use expressive flows).
- No post-hoc corrections at test time.
- No high-variance importance weights during training.
- Can reuse data from all previous rounds by reweighting past simulations.

Summary of sequential NPE variants

	SNPE-A	SNPE-B	SNPE-C
Correction	Test-time reweighting	Training-time weights	Training reparametrisation
Flow architectures?	Limited	✓	✓
Variance issues?	No	Yes	No
Reuse past rounds?	No	No	✓

Sequential SBI: The General Workflow



All three SBI flavours (NPE, NLE, NRE) can be used sequentially:

- Sequential **NPE**: SNPE-A/B/C or prior truncation.
- Sequential **NLE/NRE**: Prior truncation is straightforward; proposal-based methods also possible.

Caveat: Combining marginalisation with sequential NPE requires care — the proposal distribution affects marginal posteriors in ways not easily corrected post-training (Alsing & Wandelt, 2019).

Recently, Lyu et al. (2025) proposed **Dynamic SBI**, implementing sequential ideas in a round-free, asynchronous, parallelisable manner.

Calibration and validation of SBI posteriors

Why Diagnostics Matter

Neural networks can fail silently

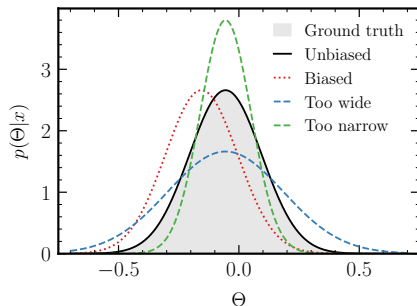
Unlike MCMC, which has convergence diagnostics (e.g. $\hat{R}$, effective sample size), neural SBI methods provide **no built-in guarantee** that the learned posterior is correct.

An undertrained network, poor architecture choice, or insufficient simulations can yield posteriors that *look* reasonable but are systematically biased or miscalibrated.

Diagnostics only require the simulator — they evaluate statistical consistency across many simulated observations.

Three levels:

- 1 **Posterior predictive checks** (necessary but not sufficient).
- 2 **Inference robustness checks** (lightweight stress tests).
- 3 **Coverage diagnostics** (rigorous calibration tests).



Posterior Predictive Checks

Idea: Draw parameters from the estimated posterior, simulate data, and compare to the observed data:

$$p(\mathbf{x} \mid \mathbf{x}_0) = \int p(\mathbf{x} \mid \boldsymbol{\Theta}) p(\boldsymbol{\Theta} \mid \mathbf{x}_0) d\boldsymbol{\Theta}$$

If simulated and observed data distributions match $\Rightarrow$ the posterior captures the data-generating process.

Necessary but not sufficient

An uninformative posterior (e.g. the prior itself) will *also* pass predictive checks while providing no useful parameter constraints. PPCs check consistency, not precision.

Recommended as a first sanity check before running more expensive coverage tests. Implemented in the `sbi` package as standard output.

Inference Robustness Checks

Lightweight stress tests that probe stability under perturbations:

- **Data masking:** Remove parts of the observed data — does the posterior change sensibly?
- **Architecture variation:** Change embedding network, number of flow layers, or hidden units. Large sensitivity $\Rightarrow$ undertrained or overfit.
- **Training seed variation:** Retrain with different random seeds. High variance $\Rightarrow$ instability.

Caveats: Variability could reflect training instability, poor hyperparameters, or model misspecification. These tests do not replace principled diagnostics, but are useful for **early warnings**.

As emphasised by Deistler et al. (2025): always inspect training loss curves, check for convergence, and compare results across multiple training runs before trusting the posterior.

Coverage Diagnostics: The Ranking Principle

Gold standard for SBI validation. Requires a calibration set of L parameter-simulation pairs:

$$\{(\Theta_i, \mathbf{x}_i)\}_{i=1}^L \sim p(\Theta, \mathbf{x})$$

For each pair, produce an estimated posterior $\hat{p}(\Theta|\mathbf{x}_i)$ and compare against the true Θ_i .

Ranking procedure: Draw M posterior samples $\{\Theta_i^{(m)}\}_{m=1}^M \sim \hat{p}(\Theta|\mathbf{x}_i)$ and compute a rank based on a projection $F : \Theta \rightarrow \mathbb{R}$:

$$R_i = \frac{1}{M} \sum_{m=1}^M \mathbb{I}[F(\Theta_i^{(m)}) < F(\Theta_i)] \in [0, 1]$$

If the posterior is well-calibrated, the ranks $\{R_i\}$ should be **uniformly distributed** across the calibration set.

Different choices of F yield different diagnostics:

- $F(\Theta) = \theta_j$ (one parameter at a time) $\Rightarrow$ **Simulation-Based Calibration (SBC)**
- $F(\Theta) = \log \hat{p}(\Theta|\mathbf{x})$ (log-probability) $\Rightarrow$ **Expected Coverage**
- $F(\Theta) = \|\Theta - \Theta_{\text{ref}}\|$ (distance to random point) $\Rightarrow$ **TARP**

Simulation-Based Calibration (SBC)

Projection: $F(\Theta) = \theta_j$ for the j -th parameter dimension.

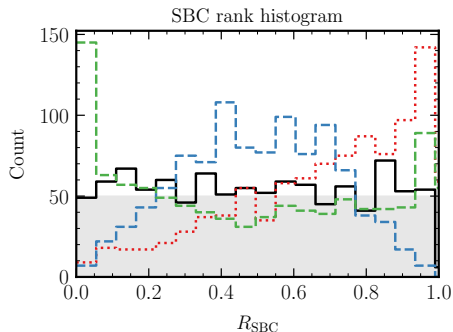
For each dimension, the histogram of ranks across all L calibration samples should be **uniform on** $[0, 1]$.

Interpretation of deviations:

Histogram shape	Diagnosis
Uniform	Well-calibrated
U-shaped	Posterior too narrow (overconfident)
Inverted-U	Posterior too broad (underconfident)
Skewed	Systematic bias (shifted mean)

Strength: Can identify which specific parameters have calibration issues.

Limitation: Cannot detect problems in parameter *correlations* if all marginals are individually well-calibrated.



Expected Coverage (z-z Plots)

Projection: $F(\Theta) = \log \hat{p}(\Theta|\mathbf{x})$, defining highest-posterior-density (HPD) credible regions.

The $(1-\alpha)$ HPD region $\Omega_{\hat{p}}(1-\alpha)$ contains all Θ with $\log \hat{p}(\Theta|\mathbf{x}) \geq c_\alpha$. The empirical coverage is:

$$1 - \hat{\alpha} = \mathbb{E}_{p(\mathbf{x}, \Theta)} [\mathbb{I}(\Theta \in \Omega_{\hat{p}(\Theta|\mathbf{x})}(1-\alpha))]$$

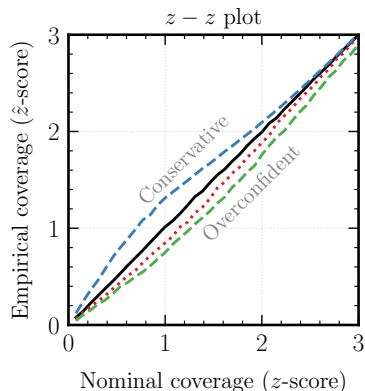
Visualisation: Plot nominal coverage $1-\alpha$ vs. empirical coverage $1-\hat{\alpha}$.

Reparametrise via z-score: $z = 1, 2, 3$ correspond to $1\sigma, 2\sigma, 3\sigma$ regions.

Deviation from diagonal	Diagnosis
On diagonal	Perfect calibration
Above diagonal	Conservative (posterior too wide)
Below diagonal	Overconfident (posterior too narrow)

Strength: Tests the *joint* distribution — can detect miscalibration in correlations.

Limitation: Tests coverage *averaged over* $\Theta \sim p(\Theta)$; some regions may be over-/under-covered.



TARP: Tests of Accuracy with Random Points

Projection: $F(\Theta) = \|\Theta - \Theta_{\text{ref}}\|$ where $\Theta_{\text{ref}} \sim \hat{p}(\Theta|\mathbf{x}_i)$ is a randomly chosen reference point.

TARP constructs credible regions as *balls centred at random reference points* with radii extending to the true parameter. By varying the reference point distribution and checking coverage, TARP can:

- Detect systematic biases that HPD-based methods might miss.
- Work with generative models where explicit density evaluations are intractable.

Computational cost of coverage diagnostics: NPE enables rapid validation (direct forward passes). NLE/NRE require MCMC per calibration point, which can be expensive. For sequential methods with prior truncation, validation remains feasible within each truncated round.

Summary of Diagnostic Tools

	PPC	SBC	Expected Cov.	TARP
Tests	Data consistency	Marginal calibration	Joint calibration	Joint calibration
Detects bias?	No	Yes	Yes	Yes
Detects correlations?	No	No	Yes	Yes
Needs density eval?	No	No	Yes	No
Cost (NPE)	Low	Medium	Medium	Medium
Cost (NLE/NRE)	Low	High	High	High

Recommendation: Always run PPC first (cheap sanity check), then SBC or expected coverage for rigorous validation. Agreement between nominal and empirical coverage is *necessary but not sufficient* for correctness — it checks consistency, not precision.

Calibration is not enough: Model mis-specification

The Elephant in the Room

All models are wrong (Box, 1976)

Every simulator is an approximation of reality. SBI assumes that the simulator faithfully represents the data-generating process.

What happens when it doesn't?

Model misspecification means the true data-generating distribution $p_{\text{true}}(\mathbf{x})$ is *not* in the family $\{p(\mathbf{x} \mid \Theta) : \Theta \in \Omega\}$ defined by our simulator.

No value of Θ can make the simulator reproduce the real data perfectly.

Examples:

- Simulator ignores a physical effect (e.g. magnetic fields, turbulence)
- Wrong noise model (assumed Gaussian, reality has heavy tails)
- Simplified geometry or discretisation artefacts
- Missing latent variables that cannot be marginalised over

Why Misspecification Is Dangerous for SBI

Under correct specification, the NPE loss $\mathcal{L} = -\mathbb{E}_{p(\mathbf{x}, \Theta)}[\log q_\Phi(\Theta|\mathbf{x})]$ converges to the true posterior as training data grows.

Under **misspecification**, the real observation $\mathbf{x}_0$ may be *out of distribution* relative to the simulated training data. The posterior $q_\Phi(\Theta|\mathbf{x}_0)$ can then be:

Overconfident:

The posterior concentrates on a narrow region that does *not* contain the true parameters. All coverage diagnostics (SBC, expected coverage) were trained on simulated data and will **pass** — they test calibration under the model, not under reality.

Biased:

The posterior mode shifts to parameters that “best explain” $\mathbf{x}_0$ *under the wrong model*, which may have no physical meaning.

Posterior predictive checks may also pass if the misspecification is subtle and the simulator can partially reproduce the data.

The core problem

Standard SBI diagnostics (SBC, expected coverage, PPC) are internal consistency checks under the model. They **cannot detect** misspecification because they never compare to real data. These diagnostics are a necessary condition for well-behaved posteriors **but not sufficient**.

How to Detect Misspecification

Two complementary strategies:

1. Posterior predictive checks against real data

Draw $\Theta \sim \hat{p}(\Theta|x_0)$, simulate $\mathbf{x}_{\text{sim}} \sim p(\mathbf{x}|\Theta)$, and compare $\mathbf{x}_{\text{sim}}$ to the *real* $\mathbf{x}_0$ using domain-specific diagnostics.

If the simulator cannot reproduce key features of the data for *any* Θ in the posterior, the model is likely misspecified.

Limitation: subjective; requires domain knowledge of what “key features” to check.

2. Distortion-driven hypothesis testing (Anau Montel, Alvey & Weniger, 2025)

Parametrise small **distortions** ϵ to the simulation model:

$$p_{\epsilon}(\mathbf{x}|\Theta) = p(\mathbf{x}|\Theta) \cdot (1 + \epsilon \cdot h(\mathbf{x}, \Theta))$$

Use NRE to test whether $\epsilon = 0$ (no misspecification) or $\epsilon \neq 0$ (distortion detected) for each distortion direction h .

This is a **simulation-based analogue of goodness-of-fit residual analysis**: scan for systematic deviations between model and data.

The Distortion Framework in Detail

Distortion-driven misspecification test (Anau Montel et al., 2025)

- 1 Define a family of distortions $\{h_k(\mathbf{x}, \Theta)\}$ — e.g. Fourier modes, wavelets, polynomial perturbations — that probe different aspects of the model.
- 2 For each distortion h_k , train a classifier (NRE) to distinguish simulations from the baseline model ($\epsilon = 0$) vs. the distorted model ($\epsilon \neq 0$).
- 3 Apply the trained classifier to the real observation $\mathbf{x}_0$. If the log-ratio is significantly different from zero, the distortion h_k is detected $\Rightarrow$ the model is misspecified in that direction.
- 4 A self-calibrating algorithm ensures that the test has correct Type I error rate without requiring separate calibration simulations.

Connections to classical methods:

- $\epsilon \rightarrow 0$ with a single distortion $\Rightarrow$ anomaly detection (is $\mathbf{x}_0$ unusual?)
- Many distortions scanned simultaneously $\Rightarrow$ goodness-of-fit residual analysis
- Two competing models $\Rightarrow$ model comparison / Bayes factor

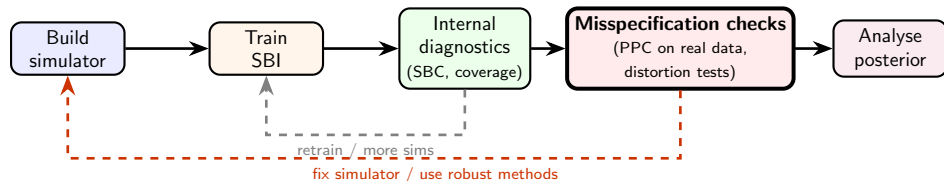
Demonstrated on the gravitational wave event GW150914, testing waveform model adequacy.

Strategies for Robust Inference Under Misspecification

Once misspecification is detected (or suspected), several strategies exist:

1. **Fix the simulator** (gold standard) Add the missing physics, improve the noise model, increase resolution. Not always feasible — the whole point of SBI is that simulators are complex.
2. **Robust summary statistics** (Huang, Bharti et al., NeurIPS 2024) Learn summaries that *down-weight* features where the model and data disagree. A regularised loss penalises statistics that increase the mismatch between simulated and real data distributions.
3. **Calibration with real data (RoPE)** (Wehenkel et al., ICML 2025) Use a small set of real observations with known ground-truth parameters to learn a correction to the posterior via optimal transport. Formalises the misspecification gap as a transport map between learned representations of real and simulated data.
4. **Widen the posterior deliberately** Treat the misspecification as additional noise. Use **deep ensembles** of independently trained SBI networks (Alvey, Contaldi & Pieroni, 2025); the spread across ensemble members captures model uncertainty beyond the statistical uncertainty from noise.

The Practical Workflow With Misspecification in Mind



Key addition to the standard workflow: After passing internal diagnostics (SBC, coverage), run misspecification checks *on real data* before trusting the posterior.

If misspecification is detected:

- First: try to improve the simulator (add missing physics)
- If not feasible: apply robust methods (robust summaries, RoPE, ensembles)
- Always: report the misspecification and quantify its impact

Summary: Model Misspecification in SBI

- ➊ **Misspecification is the rule, not the exception.** Real simulators are always approximations. SBI posteriors can be silently biased or overconfident when the simulator doesn't match reality.
- ➋ **Standard diagnostics don't help.** SBC, expected coverage, and TARP test internal consistency *under the model*. They cannot detect discrepancies between the model and reality.
- ➌ **Detection requires comparison to real data.** Posterior predictive checks against real observations are the first line of defence. Distortion-driven hypothesis tests (Anau Montel et al., 2025) provide a systematic, simulation-based framework for residual analysis.
- ➍ **Robust methods exist but are still developing.** Robust summaries, calibration with real data (RoPE), deep ensembles, and projection parameters each address different aspects of the problem. No single approach is universal.
- ➎ **The honest approach:** always test for misspecification, report any discrepancies, and quantify how they affect your conclusions. “The strength of using SBI is that it [can] obtain more accurate results even without identifying [the flaws].” But you must first *check*.

Good practices

When to Use SBI

SBI is the right tool when:

- Likelihoods are **intractable or expensive**: complex simulators, implicit marginalisation over latent variables, black-box codes.
- **Amortisation** is desired: train once, infer for many observations (real-time analysis, large catalogues).
- You want to **avoid lossy summaries** by learning compression end-to-end via embedding networks.
- Only **low-dimensional marginals** are needed in a high-dimensional nuisance space (curse of dimensionality).

When NOT to use SBI

If the likelihood is analytically tractable and cheap to evaluate, traditional methods (MCMC, nested sampling) remain the **gold standard**. They are well-established, extensively validated, and avoid the complexities of training neural networks.

SBI should not be viewed as a replacement, but as a tool for settings where traditional methods become impractical or impossible.

Choosing the SBI Algorithm

Criterion	NPE	NLE	NRE
Inference speed	Milliseconds (forward pass)	Seconds–min (MCMC)	Seconds–min (MCMC)
Training cost	Higher (flow)	Higher (flow)	Lower (classifier)
High-dim data	Embedding nets	Needs pre-compression	Embedding nets
Multiple i.i.d. obs.	Must retrain for M obs.	Factorises naturally	Factorises naturally
Marginalisation	Trivial	Trivial	Trivial
Sequential	SNPE-A/B/C or truncation	Truncation	Truncation

Rules of thumb (Deistler et al., 2025):

- **Start with NPE** — fast inference, well-studied, good default.
- Use **NLE** for high-dimensional parameter spaces or when combining multiple independent observations.
- Use **NRE** when you want an unconstrained network or when the posterior-to-prior ratio is simpler than the full density.
- Always run **diagnostics** regardless of method choice.

Practical Tips from the Literature

Simulator & Prior (Deistler et al., 2025):

- Ensure the simulator is deterministic given a random seed (reproducibility).
- Choose priors that are broad enough to cover plausible parameter ranges, but not so broad that most simulations are wasted.
- Treat the simulator as a black box: SBI methods do not need gradients through it.

Training:

- Normalise inputs to zero mean, unit variance (aids training stability, cf. Xavier initialisation from Lecture I).
- Monitor training and validation loss; use early stopping.
- For NPE with flows: Neural Spline Flows (NSF) are currently the default architecture due to their expressiveness per layer.

Validation:

- Run SBC or expected coverage tests on ~ 500 – 1000 calibration samples.
- Check robustness: retrain with different seeds, vary architecture.
- Posterior predictive checks are cheap and should always be done.

Use SBI With Caution

SBI is not plug-and-play

Successful application requires careful consideration of interconnected design choices: data compression, network architecture, training procedures, sequential strategies, and validation.

Each component introduces potential failure modes:

- Poorly chosen summary statistics that discard information.
- Undertrained networks that yield miscalibrated posteriors.
- Inappropriate sequential strategies that introduce bias.
- Insufficient simulation budget leading to poor generalisation.

Significant effort must be invested in understanding the method's assumptions and developing intuition for when the approach is working correctly.

SBI should be viewed as a **sophisticated tool requiring expertise**, not as a black-box alternative to traditional inference.

Major SBI packages:

Package	Features	Reference
sbi	NPE, NLE, NRE, sequential variants, built-in diagnostics	Tejero-Cantero et al. (2020)
swyft	Truncated marginal NRE, prior truncation	Miller et al. (2022)
lampe	Amortised posterior estimation, modular	Rozet & Delaunoy (2022)
pydelfi	Density estimation likelihood-free inference	Alsing et al. (2019)

Practical guide with code: Deistler et al. (2025), *Simulation-Based Inference: A Practical Guide* (arXiv:2508.12939), with reproducible examples at github.com/sbi-dev/sbi-practical-guide.

Lecture II Summary

- 1 **Sequential SBI** improves simulation efficiency by adaptively focusing the proposal distribution on regions consistent with $\mathbf{x}_0$. Two main strategies: prior truncation (box or correlated) and proposal-based methods (SNPE-A/B/C).
- 2 **SNPE-C** combines the strengths of its predecessors: no architecture restrictions, no post-hoc corrections, no high-variance weights, and can reuse data from all previous rounds.
- 3 **Diagnostics are essential** because neural networks can fail silently. The hierarchy is: posterior predictive checks (cheap sanity check) $\rightarrow$ SBC (marginal calibration) $\rightarrow$ expected coverage / TARP (joint calibration).
- 4 **Coverage is necessary but not sufficient:** it checks consistency, not precision. A prior-like posterior would pass coverage tests but provide no useful constraints.
- 5 **The SBI workflow** (Deistler et al., 2025) structures the process: build simulator & prior $\rightarrow$ choose components $\rightarrow$ execute SBI $\rightarrow$ test inference $\rightarrow$ analyse posterior and check for model mis-specification.
- 6 **Use SBI when** likelihoods are intractable, amortisation is needed, or you want to avoid lossy compression. Use traditional methods when likelihoods are cheap. Always validate.

References

Autoregressive models & MAF:

Germain et al. (2015). *MADE: Masked Autoencoder for Distribution Estimation*. ICML.

Papamakarios, Pavlakou & Murray (2017). *Masked Autoregressive Flow for Density Estimation*. NeurIPS.

Kingma et al. (2016). *Improved Variational Inference with Inverse Autoregressive Flow*. NeurIPS.

Coupling flows:

Dinh, Krueger & Bengio (2014). *NICE: Non-linear Independent Components Estimation*.

Dinh, Sohl-Dickstein & Bengio (2017). *Density Estimation using Real NVP*. ICLR.

Neural Spline Flows:

Durkan, Bekasov, Murray & Papamakarios (2019). *Neural Spline Flows*. NeurIPS.

Autoregressive models in other domains:

van den Oord et al. (2016). *Pixel Recurrent Neural Networks*. ICML.

van den Oord et al. (2016). *WaveNet: A Generative Model for Raw Audio*.

References

Sequential methods:

- Papamakarios & Murray (2016). SNPE-A. NeurIPS.
Lueckmann et al. (2017). SNPE-B. NeurIPS.
Greenberg, Nonnenmacher & Macke (2019). SNPE-C / APT. arXiv:1905.07488.
Miller et al. (2022). Truncated marginal NRE / swyft. JOSS.
Deistler, Gonçalves & Macke (2022). Truncated proposals. NeurIPS.
Anau Montel, Alvey & Weniger (2024). Autoregressive NRE. MNRAS 530.
Lyu et al. (2025). Dynamic SBI. arXiv:2510.13997.

Diagnostics:

- Talts et al. (2020). Simulation-based calibration. arXiv:1804.06788.
Hermans et al. (2022). Expected coverage / trust crisis. TMLR.
Lemos et al. (2023). TARP. ICML.

Practical guides:

- Deistler et al. (2025). *SBI: A Practical Guide*. arXiv:2508.12939.
Anau Montel (2025). *Basics of SBI*. MPA Lecture Notes.
Cranmer, Brehmer & Louppe (2020). *The frontier of SBI*. PNAS 117(48).

References

Detection of mis-specification:

Anau Montel, Alvey & Weniger (2025). *Tests for model misspecification in SBI: from local distortions to global model checks*. Phys. Rev. D 111, 083013.

Alvey, Contaldi & Pieroni (2025). *SBI with deep ensembles: evaluating calibration uncertainty and detecting model misspecification*. arXiv:2507.13495.

Robust methods:

Huang, Bharti et al. (2024). *Learning robust statistics for SBI under model misspecification*. NeurIPS 2024.

Wehenkel et al. (2025). *Addressing misspecification in SBI through data-driven calibration (RoPE)*. ICML 2025.

Tomaselli, Ventura & Wasserman (2025). *Robust simulation-based inference*. arXiv:2508.02404.

Reviews of mis-specification in SBI:

Kelly et al. (2025). *Simulation-based Bayesian inference under model misspecification*. arXiv:2503.12315.